Platform Engineering

a product whose customers are your own engineers

Watch the 4-min video

In one line: a platform team builds an **internal developer platform**: golden paths with service templates, self-service infrastructure with guardrails, a portal and catalog, and standard CI, deployment and observability, run **as a product** and judged by adoption, developer satisfaction and faster delivery.

1 Three weeks to launch a service

A new team needs a service. They spend three weeks creating a repository, writing a pipeline, filing a ticket for a database, learning Kubernetes manifests, building dashboards and alerts, and waiting for a security review. The next team does it again, slightly differently, so the company ends up with fifty pipelines, ten ways to deploy and no consistent monitoring. Platform engineering exists to make this take an afternoon, and to make the result consistent.

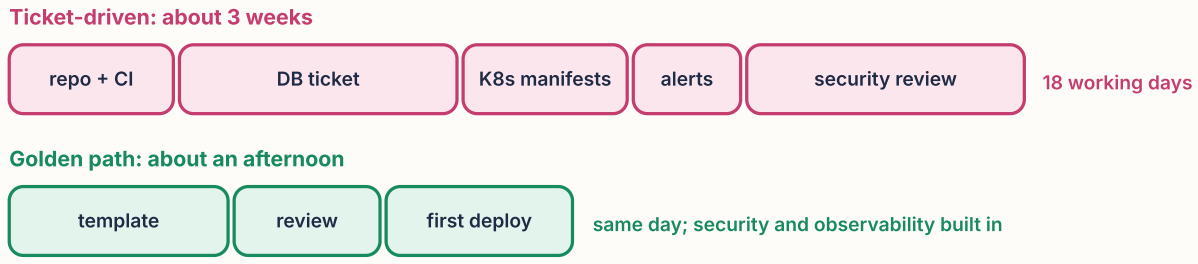


Figure 1. Illustrative timelines. The platform removes waiting on other teams, not just typing.

2 The internal developer platform

An **internal developer platform** (IDP) is the set of self-service tools and paved workflows that let product teams create services, environments and infrastructure on their own, safely, without tickets. The goal is to reduce **cognitive load**: a product team should not need to be expert in Kubernetes, Terraform, network policy and alert tuning to ship a feature. In Team Topologies terms, a **platform team** provides services to **stream-aligned** (product) teams, aiming for the "thinnest viable platform": just enough to remove real friction.

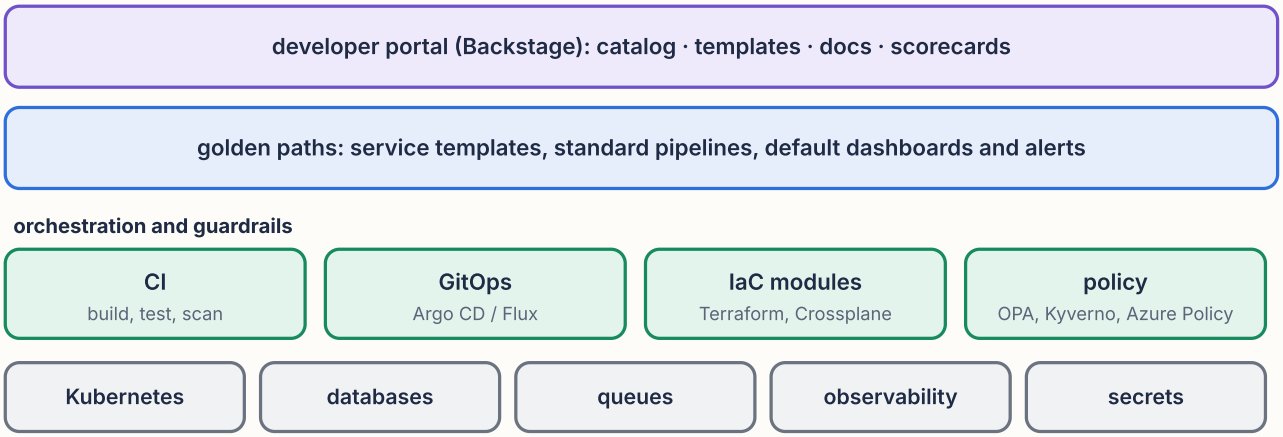


Figure 2. Layers of a typical IDP. Product teams interact with the top two layers; the platform team owns the wiring beneath.

It sits on everything earlier in this phase: Kubernetes (lessons 56–60), IaC modules (lesson 61), GitOps (lesson 62), immutable images (lesson 63) and cost guardrails (lesson 64). The platform's job is to combine them into a few well-lit paths.

3 Golden paths

A **golden path** (Spotify's term; Netflix says **paved road**) is the recommended, supported way to build and run a service: a standard language template, CI pipeline, deployment method, logging, metrics, tracing, security scanning and runbook. It's **opinionated but optional**: teams may leave it when they truly need to, but then they own what they build. Staying on it is simply the easiest choice, because it's faster and comes with support. Mandating instead of attracting tends to produce workarounds.

4 Templates make golden paths real

A **service template** (Backstage Software Templates, or Cookiecutter / Yeoman behind a portal) asks a few questions (name, owner, language, needs a database?) and creates, in one step, a repository with a skeleton app, Dockerfile, CI pipeline, Helm chart or IaC config, dashboards, alert rules, and the catalog entry, all wired together and following company standards. Two subtleties:

- Templates generate code once; updates to the template don't reach existing services by themselves. Keep shared logic in **versioned dependencies** (a shared CI pipeline template, a base Helm chart, base images) so a fix reaches every service by a version bump, possibly automated with Renovate or Dependabot.
- **Scorecards** in the portal show which services meet current standards (owner set, alerts defined, runtime version supported), which is how you migrate the fleet gradually.

5 Self-service infrastructure and the portal

Teams request a database, queue or bucket through the portal or a pull request, using **pre-approved IaC modules** or Kubernetes-style APIs (Crossplane compositions) with guardrails built in: private networking only, encryption on, backups configured, tags and budget set, sizes from an allowed list. **Policy as code** (OPA Gatekeeper, Kyverno, Azure Policy, Sentinel) blocks what modules can't express: no public storage accounts, no privileged containers, mandatory owner labels. Guardrails let teams move fast without accidentally creating a public database or a $50,000 GPU cluster.

A **developer portal** such as **Backstage** (created at Spotify, now a CNCF project) ties it together: a **software catalog** of services, APIs, owners and dependencies (each repository has a `catalog-`

`info.yaml`), the templates, technical documentation (TechDocs) built from Markdown in each repo, and plugins linking to CI runs, deployments, dashboards, costs and on-call. Commercial alternatives (Port, Cortex, OpsLevel) offer the same model.

6 Run the platform as a product

The platform's customers are internal, but the discipline is the same as for any product: user research with developers, a roadmap, documentation, support, deprecation policies and measured outcomes. A platform nobody wants to use has failed, however clever it is.



Figure 3. What to measure. DORA's four keys (deployment frequency, lead time, change failure rate, recovery time) show whether the platform improves delivery.

	Ticket-driven ops	Platform (self-service)
Getting a database	ticket, days of waiting	PR or portal form, minutes, policy-checked
Consistency	depends on who handled the ticket	same modules and templates for everyone
Ops team scaling	grows with the number of requests	grows with the number of capabilities
Security / compliance	manual reviews at the end	guardrails and defaults from the start

WORKED EXAMPLE: A SERVICE TEMPLATE FROM REPO TO FIRST DEPLOY

A developer opens Backstage, picks the **Python API** template, and enters a name (`invoice-api`), an owning team and "needs Postgres: yes". Within minutes the platform creates a repository with a FastAPI skeleton, health endpoints and tests; a CI pipeline that builds, scans and pushes an image; a Helm chart plus an entry in the GitOps config repo, so Argo CD deploys it to **dev** (lesson 62); a Postgres database from the standard Terraform module with credentials in Key Vault; a Grafana dashboard for the four golden signals; and default alerts routed to the team's on-call. The service appears in the catalog with its owner, docs and links. The developer merges a first feature that afternoon, and it's running in dev the same day. Promotion to production follows the same PR flow as every other service.

COMMON MISTAKES

- Building a platform nobody asked for, then mandating it.
- A portal that is just a list of links, without real self-service behind it.
- Templates that fork once and rot, with no versioned shared parts.
- Measuring the platform by features shipped instead of adoption and delivery speed.

7 Key terms

Internal developer platform: self-service tools and paved workflows for engineers

Service template: generator for a fully wired new service

Golden path / paved road: supported, recommended way to build and run a service

Developer portal: UI for catalog, templates, docs (e.g. Backstage)

Guardrails: defaults and policies that prevent unsafe configurations

Cognitive load: what a team must know to deliver; platforms reduce it

8 Interview questions

Q1. What does a platform team provide? BASIC

An internal developer platform: golden paths with service templates, self-service infrastructure with built-in guardrails, a developer portal and catalog, and standard CI, deployment and observability, run as a product for the company's engineers.

Q2. How do you know it works? BASIC

Adoption of the golden path, developer satisfaction, and delivery metrics such as time to first deploy, lead time and deployment frequency.

Q3. What is a golden path? BASIC

The recommended, supported way to build and run a service, bundling template, pipeline, deployment, observability and security; optional but the easiest choice.

Q4. Teams keep creating public storage buckets by accident. What should the platform do? SCENARIO

Provide a storage module that is private by default and enforce a policy-as-code rule that blocks public access, so the safe option is automatic.

Q5. You updated the service template's CI. How do the 200 existing services get it? SCENARIO

Templates don't propagate; keep CI as a versioned shared pipeline referenced by each repo, roll out version bumps automatically, and track compliance with scorecards.

Q6. Isn't a platform team just a renamed ops team? TRICKY

No: ops teams fulfil requests; a platform team builds self-service products so requests aren't needed, and measures success by product teams' outcomes.

Q7. Should you force every team onto the platform? TRICKY

Prefer attraction: make the golden path the easiest choice and support it well; allow exits with ownership, and use mandates only for real guardrails like security policies.

Next: SD-V-021 · Workflow Engines backendroadmap.com